



数据库系统课程实验报告

实验名称:	数据库的完整性
实验日期:	2025.11.18
实验地点:	文宣楼 B308
提交日期:	2025.11.25

学号:	37220232203780
姓名:	马鑫
专业年级:	2023 级数字媒体技术
学年学期:	第三学年第一学期

1.实验目的

- 掌握数据库的特点（字体：华文仿宋，字号：四号，下同）
- 理解并掌握关系数据库完整性的运行机制
 - 完整性约束定义>完整性约束检查>违约处理
- 理解并掌握关系数据库完整性主要约束类型及其含义和作用
 - PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK
- 理解并掌握关系数据库完整性定义、修改、删除和重命名的方法
 - CREATE TABLE, ALTER TABLE
- 熟练掌握 openGauss 下通过系统表 pg_constraint 查看完整性信息的方法
- 熟练掌握 openGauss 下通过查看表结构来查看主外码信息的方法
- 熟练掌握 openGauss 下通过查看完整性约束定义的方法

2.实验内容和步骤

该部分的写作格式示例如下：

(1) 创建两张表：雇员表 Emp 和工作表 Work，它们的表结构如下：

Emp 表

字段	含义	数据类型	是否空
Eid	雇员编号	定长字符型，长度为 5	否
Ename	雇员姓名	变长字符型，长度为 10	/
WorkID	工作编号	定长字符，长度为 3	/
Salary	工资	数值型，总长度为 8，包括两位小数	/
Phone	电话号码	定长字符型，长度为 11	否

Work 表

字段	含义	数据类型	是否空
WorkID	工作编号	定长字符，长度为 3	否
LowerSalary	最低工资	数值型，总长度为 8，包括两位小数	/
UpperSalary	最高工资	数值型，总长度为 8，包括两位小数	/

步骤如下：

```
employ=> CREATE TABLE Emp
employ-> (
employ(> Eid CHAR(5) NOT NULL,
employ(> Ename VARCHAR(10),
employ(> WorkID CHAR(3),
employ(> Salary NUMERIC(8,2),
employ(> Phone CHAR(11) NOT NULL
employ(> );
CREATE TABLE
```

```
employ=> CREATE TABLE Work
employ-> (
employ(> WorkID CHAR(3) NOT NULL,
employ(> LowerSalary NUMERIC(8,2),
employ(> UpperSalary NUMERIC(8,2)
employ(> );
CREATE TABLE
```

(2) 分别为两张表插入如下数据，查看插入操作是否成功

雇员表数据：{('10001' , ' Smith ' , ' 001 ' ,
2000 , ' 13800010001 '), (' 10001 ' , ' Jonny ' , ' 001 ' ,
3000 , ' 13600010002 '), (' 10002 ' , ' Mary ' , ' 002 ' , 2500 , ' 13800020002 ') }

工作表数据：{ ('001 ' , 1000, 5000), (' 002 ' , 2000, 8000) }

步骤如下：

1. 插入数据。

```
employ=> INSERT INTO Emp VALUES('10001','Smith','001',2000,'13800010001'),('10001','Jonny','001',3000,'13600010002'),('10002','Mary','002',2500,'13800020002');
INSERT 0 3

employ=> INSERT INTO Work VALUES('001',1000,5000),('002',2000,8000);
INSERT 0 2
```

2. 通过查询查看是否插入成功。

```
employ=> SELECT * FROM Emp;
  eid | ename | workid | salary |      phone
-----+-----+-----+-----+-----
10001 | Smith | 001    | 2000.00 | 13800010001
10001 | Jonny | 001    | 3000.00 | 13600010002
10002 | Mary  | 002    | 2500.00 | 13800020002
(3 rows)
```

```
employ=> SELECT * FROM Work;
 workid | lowersalary | uppersalary
-----+-----+-----
001     | 1000.00    | 5000.00
002     | 2000.00    | 8000.00
(2 rows)
```

插入成功。

(3) 修改雇员表的结构，设置 Eid 为主码，主码名称为 eid_pk，查看该操作是否成功。若不成功，请说明原因并思考如何处理才能成功添加主码约束。要求：所有约束都要显式给出约束名，不可由系统默认，因为删除约束时需要用到约束名

步骤如下：

1. 添加主码。

```

employ=> ALTER TABLE Emp ADD CONSTRAINT eid_pk PRIMARY KEY(Eid);
NOTICE: ALTER TABLE / ADD PRIMARY KEY will create implicit index "eid_pk" for table "emp"
ERROR: could not create unique index "eid_pk"
DETAIL: Key (eid)=(10001) is duplicated.

```

失败，由于有重复的 Eid，导致不满足 UNIQUE 唯一性要求。

删除或修改重复的 Eid 其中之一。

2. 修改雇员表。

```

employ=> UPDATE Emp SET Eid='10003' WHERE Ename='Jonny';
UPDATE 1
employ=> SELECT * FROM Emp;
  eid | ename | workid | salary |   phone
-----+-----+-----+-----+-----
 10001 | Smith |    001 | 2000.00 | 13800010001
 10002 | Mary  |    002 | 2500.00 | 13800020002
 10003 | Jonny |    001 | 3000.00 | 13600010002
(3 rows)

```

3. 再次添加主键。

```

employ=> ALTER TABLE Emp ADD CONSTRAINT eid_pk PRIMARY KEY(Eid);
NOTICE: ALTER TABLE / ADD PRIMARY KEY will create implicit index "eid_pk" for table "emp"
ALTER TABLE

```

```

employ=> \d Emp;
          Table "employ.emp"
  Column |          Type          | Modifiers
-----+-----+-----+-----
  eid    | character(5)           | not null
  ename  | character varying(10)  |
  workid | character(3)           |
  salary | numeric(8,2)           |
  phone  | character(11)          | not null
Indexes:
    "eid_pk" PRIMARY KEY, btree (eid) TABLESPACE pg_default

```

(4) 将 eid 为主码的约束名 eid_pk 改为 pk_eid

步骤如下：


```

employ=> ALTER TABLE Emp DROP CONSTRAINT eid_pk;
ALTER TABLE
employ=> ALTER TABLE Emp ADD CONSTRAINT pk_eid PRIMARY KEY(Eid);
NOTICE: ALTER TABLE / ADD PRIMARY KEY will create implicit index "pk_eid" for table "emp"
ALTER TABLE

```

(5) 设置雇员表中的 phone 字段取唯一值，查看该操作是否成功？

若不成功说明原因

步骤如下：

```

employ=> ALTER TABLE Emp ADD CONSTRAINT uk_emp_phone UNIQUE (Phone);
NOTICE: ALTER TABLE / ADD UNIQUE will create implicit index "uk_emp_phone" for table "emp"
ALTER TABLE

```

```

employ=> \d Emp
               Table "employ.emp"
  Column |          Type          | Modifiers
-----+-----+-----
 eid    | character(5)           | not null
 ename   | character varying(10)  |
 workid  | character(3)           |
 salary  | numeric(8,2)           |
 phone   | character(11)          | not null
Indexes:
    "pk_eid" PRIMARY KEY, btree (eid) TABLESPACE pg_default
    "uk_emp_phone" UNIQUE CONSTRAINT, btree (phone) TABLESPACE pg_default

```

(6) 给雇员表添加一条新记录('10003' , ' Amy' , ' 002' ,
3000, ' 13800020003')，查看执行结果

步骤如下：

```

employ=> INSERT INTO Emp VALUES('10003','Amy','002',3000,'13800020003');
ERROR:  duplicate key value violates unique constraint "pk_eid"
DETAIL:  Key (eid)=(10003) already exists.

```

此处由于主键 Eid 重复了，插入失败，将 Eid 改为 '10004' 再次插入。

```

employ=> INSERT INTO Emp VALUES('10004','Amy','002',3000,'13800020003')
INSERT 0 1

```

```

employ=> SELECT * FROM Emp;
  eid | ename | workid | salary |      phone
-----+-----+-----+-----+-----
 10001 | Smith | 001    | 2000.00 | 13800010001
 10002 | Mary  | 002    | 2500.00 | 13800020002
 10003 | Jonny | 001    | 3000.00 | 13600010002
 10004 | Amy   | 002    | 3000.00 | 13800020003
(4 rows)

```

(7) 设置工作表的 WorkID 为主码

步骤如下:

```

employ=> ALTER TABLE Work ADD CONSTRAINT pk_workid PRIMARY KEY(workid);
NOTICE: ALTER TABLE / ADD PRIMARY KEY will create implicit index "pk_workid" for table "work"
ALTER TABLE

```

(8) 修改雇员表, 设置雇员表的 WorkID 字段为外码, 它引用工作表中的 WorkID 字段, 查看操作是否成功? 若不成功说明原因

步骤如下:

```

employ=> ALTER TABLE Emp ADD CONSTRAINT fk_emp_workid FOREIGN KEY(workid) REFERENCES Work(workid);
ALTER TABLE

```

操作成功。

(9) 给雇员表添加一条新记录('10003' , ' Amy' , '003' , 3000, '13800020003'), 查看操作是否成功? 若不成功说明原因

步骤如下:

```

employ=> INSERT INTO Emp VALUES('10005','Amy','003',500,'13800020003');
ERROR:  duplicate key value violates unique constraint "uk_emp_phone"
DETAIL:  Key (phone)=(13800020003) already exists.

```

操作失败, 因为 phone 也存在, 不满足 UNIQUE 唯一性。

(10) 在雇员表中, 设置雇员工资必须大于或等于 1000。查看操作是否成功? 若不成功说明原因

步骤如下:

```
employ=> ALTER TABLE Emp ADD CONSTRAINT ck_emp_salary_range CHECK(Salary>=1000);  
ALTER TABLE
```

```
employ=> \d Emp;  
  
          Table "employ.emp"  
  Column |          Type          | Modifiers  
-----+-----+-----  
 eid     | character(5)           | not null  
 ename   | character varying(10)  |  
 workid  | character(3)           |  
 salary  | numeric(8,2)           |  
 phone   | character(11)          | not null  
Indexes:  
    "pk_eid" PRIMARY KEY, btree (eid) TABLESPACE pg_default  
    "uk_emp_phone" UNIQUE CONSTRAINT, btree (phone) TABLESPACE pg_default  
Check constraints:  
    "ck_emp_salary_range" CHECK (salary >= 1000::numeric)  
Foreign-key constraints:  
    "fk_emp_workid" FOREIGN KEY (workid) REFERENCES work(workid)
```

(11) 给雇员表添加一条新记录

('10003' , ' Robert' , '002' , 500 , '13800020003'), 查看执行操作是否成功? 若不成功说明原因

步骤如下:

```
employ=> INSERT INTO Emp VALUES('10005','Robert','002',500,'13800020003');  
ERROR: new row for relation "emp" violates check constraint "ck_emp_salary_range"  
DETAIL:  Failing row contains (10005, Robert, 002, 500.00, 13800020003).
```

不成功, 因为 Salary=500 不满足 CHECK 约束的 Salary $\geq$ 1000。

(12) 在工作表中, 设置其最低工资不超过最高工资

步骤如下:

```
employ=> ALTER TABLE Work ADD CONSTRAINT ck_work_salary_range CHECK(LowerSalary<=UpperSalary);  
ALTER TABLE
```

(13) 给工作表添加一条新记录('002' , 4000, 3000), 查看操作是否成功? 若不成功说明原因

步骤如下:


```
employ=> INSERT INTO Work VALUES('002',4000,3000);
ERROR:  new row for relation "work" violates check constraint "ck_work_salary_range"
DETAIL:  Failing row contains (002, 4000.00, 3000.00).
```

因为 LowerSalary=4000>UpperSalary=3000 不满足 CHECK 约束条件

$$\text{LowerSalary} \leq \text{UpperSalary}.$$

(14)通过查看 openGauss 的系统表 pg_constraint 了解表上的约束

步骤如下：

[illegible][illegible]

```

pk_eid          |          |          | 16923 | p | f | f | t |          | 16924 |          | 16934 |          | 0 |
              |          |          |      | t |   | t | f | f |          | {1} |          |          |          |
              |          |          |          |          |          |          |          |          |          |          |

uk_emp_phone    |          |          | 16923 | u | f | f | t |          | 16924 |          | 16936 |          | 0 |
              |          |          |      | t |   | t | f | f |          | {5} |          |          |          |
              |          |          |          |          |          |          |          |          |          |          |

pk_workid       |          |          | 16923 | p | f | f | t |          | 16927 |          | 16938 |          | 0 |
              |          |          |      | t |   | t | f | f |          | {1} |          |          |          |
              |          |          |          |          |          |          |          |          |          |          |

```

```

fk_emp_workid   |          |          | 16923 | f | f | f | t |          | 16924 |          | 16938 |          | 16927 | a
              | a          | u          |      | t |   | t | f | f |          | {3} | {1} | {1054} | {1054} | {
1054}          |          |          |          |          |          |          |          |          |          |

ck_emp_salary_range |          |          | 16923 | c | f | f | t |          | 16924 |          |          |          | 0 |
              |          |          |      | t |   | f | f | f |          | {4} |          |          |          |
              |          | {OPEXPR :opno 1757 :opfuncid 1721 :opresulttype 16 :opretset false :opcollid 0 :inputcollid 0 :args ({VAR :varno 1 :varattno
4 :vartype 1700 :vartypmod 524294 :varcollid 0 :varlevelsup 0 :varnoold 1 :varoattno 4 :location 57} {FUNCEXPR :funcid 1740 :funcresulttype 1700 :fun
cretset false :funcformat 2 :funccollid 0 :inputcollid 0 :args ({CONST :consttype 23 :consttypmod -1 :constcollid 0 :constlen 4 :constbyval true :con
stisnull false :ismaxvalue false :location 65 :constvalue 4 [-24 3 0 0 0 0 0 0] :cursor_data :row_count 0 :cur_dno -1 :is_open false :found false
:not_found false :null_open false :null_fetch false}) :location -1 :refsynoid 0}) :location 63}

              | (salary >= (1000)::numeric)

ck_work_salary_range |          |          | 16923 | c | f | f | t |          | 16927 |          |          |          | 0 |
              |          |          |      | t |   | f | f | f |          | {2,3} |          |          |          |
              |          | {OPEXPR :opno 1755 :opfuncid 1723 :opresulttype 16 :opretset false :opcollid 0 :inputcollid 0 :args ({VAR :varno 1 :varattno 2 :
vartype 1700 :vartypmod 524294 :varcollid 0 :varlevelsup 0 :varnoold 1 :varoattno 2 :location 59} {VAR :varno 1 :varattno 3 :vartype 1700 :vartypmod
524294 :varcollid 0 :varlevelsup 0 :varnoold 1 :varoattno 3 :location 72}) :location 70}

```

```

              | (lowersalary <= uppersalary)

(8 rows)

```

(15) 通过 gsql 命令 \d+ table_name 查看该表上的约束定义
步骤如下:

```

employ=> \d+ Emp;

```

Column	Type	Modifiers	Storage	Stats target	Description
eid	character(5)	not null	extended		
ename	character varying(10)		extended		
workid	character(3)		extended		
salary	numeric(8,2)		main		
phone	character(11)	not null	extended		

Indexes:

- "pk_eid" PRIMARY KEY, btree (eid) TABLESPACE pg_default
- "uk_emp_phone" UNIQUE CONSTRAINT, btree (phone) TABLESPACE pg_default

Check constraints:

- "ck_emp_salary_range" CHECK (salary >= 1000::numeric)

Foreign-key constraints:

- "fk_emp_workid" FOREIGN KEY (workid) REFERENCES work(workid)

Has OIDs: no

Options: orientation=row, compression=no

```

employ=> \d+ Work;

```

Column	Type	Modifiers	Storage	Stats target	Description
workid	character(3)	not null	extended		
lowersalary	numeric(8,2)		main		
uppersalary	numeric(8,2)		main		

Indexes:

- "pk_workid" PRIMARY KEY, btree (workid) TABLESPACE pg_default

Check constraints:

- "ck_work_salary_range" CHECK (lowersalary <= uppersalary)

Referenced by:

- TABLE "emp" CONSTRAINT "fk_emp_workid" FOREIGN KEY (workid) REFERENCES work(workid)

Has OIDs: no

Options: orientation=row, compression=no

(16) 删除雇员表的所有约束，包括主码约束、外码约束和其他约束

步骤如下：

```
employ=> ALTER TABLE Emp DROP CONSTRAINT fk_emp_workid;
ALTER TABLE
employ=> ALTER TABLE Emp DROP CONSTRAINT uk_emp_phone;
ALTER TABLE
employ=> ALTER TABLE Emp DROP CONSTRAINT ck_emp_salary_range;
ALTER TABLE
employ=> ALTER TABLE Emp DROP CONSTRAINT pk_eid;
ALTER TABLE
employ=> \d+ Emp
```

Column	Type	Modifiers	Storage	Stats target	Description
eid	character(5)	not null	extended		
ename	character varying(10)		extended		
workid	character(3)		extended		
salary	numeric(8,2)		main		
phone	character(11)	not null	extended		

Has OIDs: no
Options: orientation=row, compression=no

(17) 删除工作表所有约束，包括主码约束

步骤如下：

```
employ=> ALTER TABLE Work DROP CONSTRAINT ck_work_salary_range;
ALTER TABLE
employ=> ALTER TABLE Work DROP CONSTRAINT pk_workid;
ALTER TABLE
employ=> \d+ Work
```

Column	Type	Modifiers	Storage	Stats target	Description
workid	character(3)	not null	extended		
lowersalary	numeric(8,2)		main		
uppersalary	numeric(8,2)		main		

Has OIDs: no
Options: orientation=row, compression=no

3.实验总结

3.1 完成的工作

完成实验内容，完成完整性约束的添加、修改与删除。

3.2 对实验的认识

通过本实验增加了数据库完整性的理解，学习了主要通过 ALTER 对基本表约束的管理。

思考题：openGauss 实现完整性规则的机制是什么？在 SQL 语

句中实现完整性规则的常见约束有哪些？各自适用什么业务场景？

答：主要通过约束、触发器、规则等多种方式来实现完整性规则。
主键约束 PRIMARY KEY，适用于需要唯一标识的业务实体；外键约束 FOREIGN KEY，适用于数据关联关系；唯一约束 UNIQUE，适用于防止数据重复；检查约束 CHECK，适用于业务规则验证；非空约束 NOT NULL，默认值 DEFAULT，适用于保障数据完整性。

3.3 遇到的困难及解决方法

无。